CDN for Private Content

fast for everyone, readable only by payers

Watch the 4-min video

In one line: keep the origin private so only the CDN can read it, have your app issue short-lived signed URLs or signed cookies after checking access, and let the CDN validate them at the edge; design cache keys so everyone shares cached copies, prefer versioned URLs to purges, and protect the origin with a shield layer.

1 Fast, and leaked

An online course platform serves its videos through a CDN and they load fast worldwide. Then someone posts a lesson's URL on a forum and thousands of people who never paid start watching, at your bandwidth cost. Or someone finds the bucket URL behind the CDN and downloads everything directly. A CDN makes content *fast*; making private content fast *and* private needs three things: a locked origin, proof of access checked at the edge, and caching that still works when every user has different credentials.

2 Lock down the origin

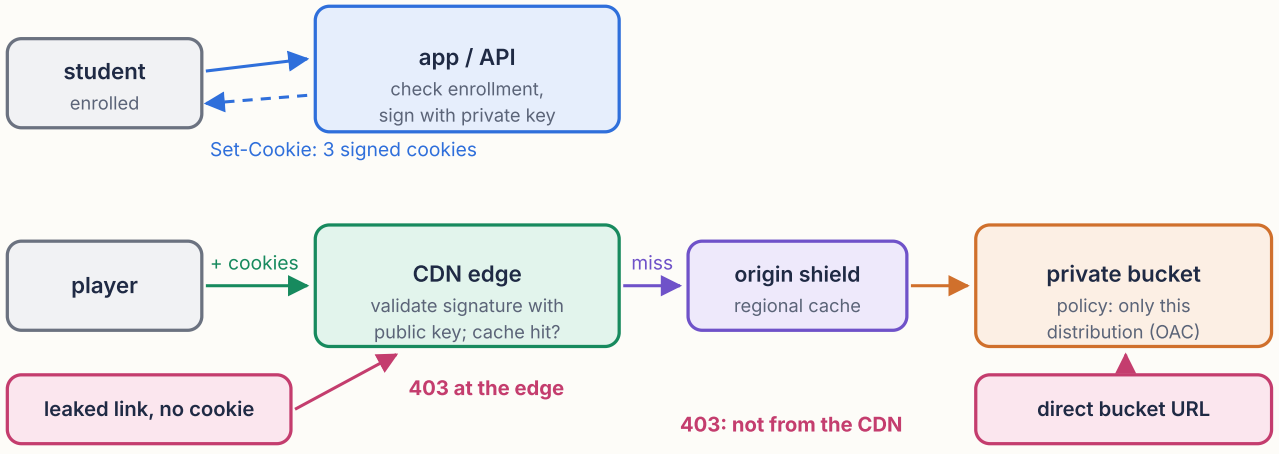


Figure 1. The app proves access by signing; the edge verifies without calling the app; the bucket accepts only the CDN.

The CDN fetches from an **origin**, usually a bucket. If the bucket is reachable directly, any access check at the edge can be skipped. On AWS, block all public access and use CloudFront **Origin Access Control** (OAC): CloudFront signs its requests to S3 with SigV4, and the bucket policy allows the `cloudfront.amazonaws.com` service principal only when `AWS:SourceArn` is your distribution (OAC replaces the older origin access identity and also supports SSE-KMS). On Azure, Front Door Premium can reach a storage origin over **Private Link** so the account needs no public endpoint. For a custom origin, accept only the CDN's IP ranges plus a secret header the CDN adds, or use mutual TLS.

3 Signed URLs and signed cookies

The CDN, not your app, checks every request. Your app signs a **policy** (which resource, until when, optionally from which IP) with a **private key**; the CDN holds the matching **public key** (CloudFront: a *trusted key group*) and verifies the signature locally, so validation adds no round trip to your servers. A

canned policy covers one exact URL and an expiry; a **custom** policy allows wildcards (`/courses/sd/*`), a start time and an IP range.

	Signed URL	Signed cookies
Carries	query string: `Expires` or `Policy`, `Signature`, `Key-Pair-Id`	cookies: `CloudFront-Policy` (or `-Expires`), `CloudFront-Signature`, `CloudFront-Key-Pair-Id`
Covers	one file (or a wildcard, with a custom policy)	many files at once, e.g. every manifest and segment of a course
Best for	a single download, an email link, clients that can't hold cookies	HLS/DASH streaming, galleries, whole private sections
Needs	nothing; works across domains	CDN on a subdomain of your app (cookie domain `.example.com`), Secure + HttpOnly
Leak risk	URL ends up in logs, chats, history	cookies can be exported, but don't appear in links

For streaming, signed URLs are awkward: the player requests hundreds of segment URLs listed in manifests you would have to rewrite per user. Signed cookies cover them all. (If a request carries both, CloudFront uses the signed URL.) Other CDNs offer the same idea under other names: Cloudflare token authentication (HMAC checked by a WAF rule), Akamai EdgeAuth tokens. These CDN signatures are different from storage pre-signed URLs (lesson 5): they're validated at the edge, so the content stays cacheable.

4 Cache keys

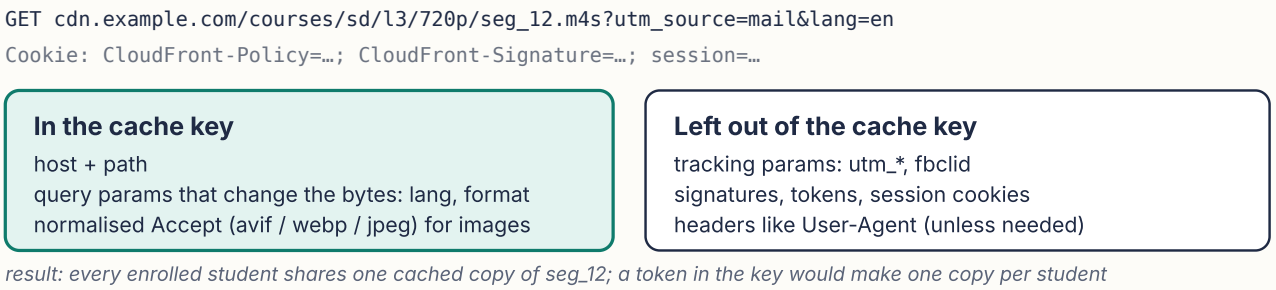


Figure 2. The cache key decides what counts as "the same object". Keep per-user and tracking data out; keep content-changing inputs in.

By default the key is roughly host + path. Two opposite failure modes: **too much** in the key (tracking parameters, tokens, cookies, `User-Agent`) and the same bytes are cached thousands of times with a terrible hit ratio; **too little** (ignoring a `lang` or format parameter that changes the response) and users get the wrong version. On CloudFront this is a *cache policy* (which headers, cookies and query strings are in the key) separate from an *origin request policy* (what is forwarded to the origin). Signed cookies are validated separately and don't need to be in the key, so all students share one cached segment.

5 Purge or version?

- **Purge / invalidation**: remove paths (or tags, on some CDNs) from every edge. Useful for emergencies (a leaked file, a legal takedown), but it takes time to propagate, is rate-limited and may be billed (CloudFront: first 1,000 paths a month free), and it doesn't clear browser caches.
- **Versioned URLs** (the default): put a version or content hash in the name, `app.3f9a2c.js`, `/v7/l3/seg_12.m4s`. Changed content gets a new URL, so it is a guaranteed miss everywhere at once;

old objects just expire. Cache versioned files for a year (`max-age=31536000, immutable`) and give only the few entry points (HTML pages, VOD master manifests if they change) short TTLs.

6 Origin shield and multi-CDN

A CDN has hundreds of edge locations; without help, a new video's first segment can be fetched from the origin once per edge. An **origin shield** (CloudFront Origin Shield, Cloudflare Tiered Cache, Akamai tiered distribution) routes all edge misses through one regional cache layer, and request collapsing turns simultaneous misses into one origin fetch, so the origin sees one request instead of hundreds. **Multi-CDN** spreads traffic over two or more providers for resilience and price: DNS-based steering (weighted, latency- or health-based), or player-side switching on errors (HLS and DASH both define *content steering*). The cost is keeping config in sync: cache keys, TLS certificates, purge APIs, logging, and especially token formats, since each CDN validates its own signatures; your app must issue a token per CDN.

7 Example: paid course videos

WORKED EXAMPLE: SERVING PAID COURSE VIDEOS WITH SIGNED COOKIES

The app runs at `example.com`, the CDN at `cdn.example.com`, the bucket is private with OAC. A student opens lesson 3; the API checks enrollment and sets three CloudFront cookies (`Domain=.example.com; Secure; HttpOnly`) signed over a custom policy: resource `https://cdn.example.com/courses/sd/*`, expiry now + 2 hours. The player loads `/courses/sd/l3/master.m3u8` and its segments; each request carries the cookies, the edge validates them, and the cache key is just the path, so the second student's requests are edge hits. Every hour the player calls `/refresh-cdn-cookies`, which re-checks enrollment, so a refund cuts access within an hour.

The forum link fails with 403 (no cookies); the raw bucket URL fails with 403 (only CloudFront may read). A determined user can still copy their own cookies for up to 2 hours; if that matters, add DRM (Widevine / FairPlay, with per-user licences and concurrent-stream limits) and forensic watermarking.

TRADE-OFFS

- Short expiry limits leaks but needs a refresh flow; long expiry is simple but leaks last longer.
- IP-restricted policies stop link sharing but break mobile users who change networks.
- Signed cookies need the CDN on your domain; signed URLs work anywhere but bloat manifests and leak through links.
- Multi-CDN improves resilience and pricing leverage at the cost of duplicated config and token logic.

COMMON MISTAKES

- Leaving the bucket readable, so the CDN's checks can be bypassed.
- Putting tokens, session cookies or tracking parameters in the cache key: near-zero hit ratio.
- Relying on purges for every deploy instead of versioned file names.
- Storing the CDN signing private key in the client or in a public repo.

8 Key terms

Origin Access Control: only the CloudFront distribution may read the bucket

Canned vs custom policy: one URL + expiry vs wildcards, start time, IP

Versioned URL: new content gets a new URL, so no purge is needed

Signed URL / signed cookie: CDN-validated proof of access with an expiry

Cache key: inputs that identify one cached object

Origin shield: extra cache tier that funnels edge misses to the origin

9 Interview questions

Q1. How do you serve private content via a CDN? BASIC

Keep the origin private (reachable only from the CDN) and have the CDN validate short-lived signed URLs or signed cookies that your app issues after checking access.

Q2. Signed URLs or signed cookies? BASIC

Signed URLs suit single files and clients without cookies; signed cookies cover many files at once, like video segments, without rewriting manifests.

Q3. Your CDN hit ratio for a course platform is 5%. What do you check? SCENARIO

The cache key: per-user tokens, session cookies or tracking parameters in it create one copy per user; key on path (plus real variant inputs) and validate tokens separately.

Q4. A file was published by mistake and must disappear now. What do you do? SCENARIO

Remove it at the origin, invalidate the path on every CDN, and revoke access tokens if needed; for routine updates use versioned URLs instead of purges.

Q5. A new video launches and the origin is hammered on release. Fix? SCENARIO

Add an origin shield / tiered cache with request collapsing, and optionally pre-warm popular segments, so each object is fetched from the origin once.

Q6. Why not use S3 pre-signed URLs for every video segment? TRICKY

Each viewer gets unique URLs, so the CDN can't share cached copies and every manifest must be rewritten per user; CDN-validated cookies keep URLs identical and cacheable.

Q7. What breaks when you add a second CDN? TRICKY

Token formats and signing keys, cache-key rules, purge APIs, TLS certificates and logging all differ per provider, so access control and invalidation must be implemented for each.

Next: SD-O-012 · Data Lifecycle backendroadmap.com